Retail Sales Data Pipeline and Analytics Dashboard

Author: Mykhailo Stom

Demo agenda

1. Slides, explanation of the created solution
2. Dashboard showcase

Summary

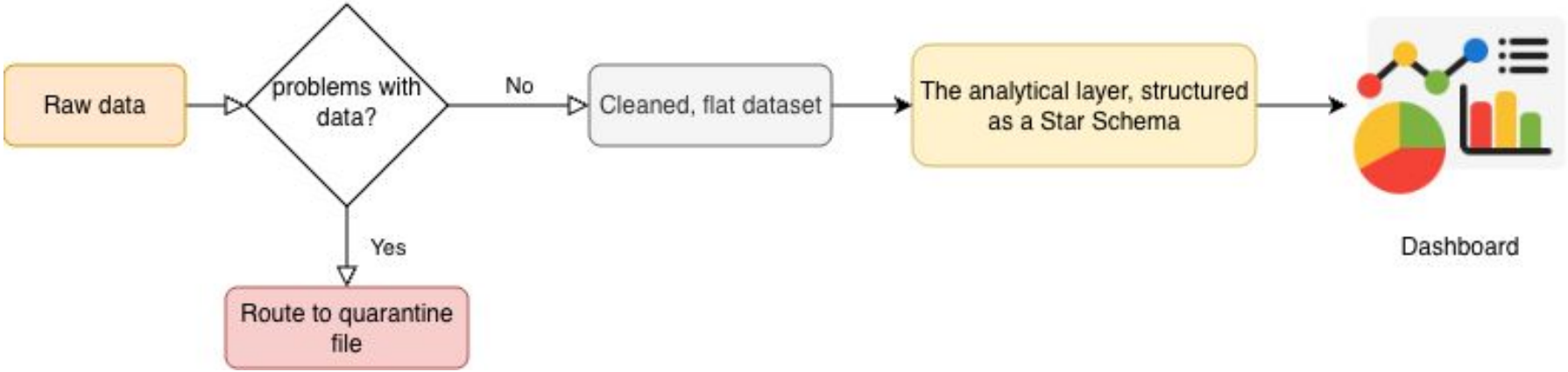
This project prototypes the data engineering and analytics solution for processing, cleaning, and visualizing retail sales data. It uses a Medallion Data Architecture to progressively refine raw CSV data into a highly optimized Star Schema, which is then queried by a Streamlit dashboard.

Tech Stack

Solution is done entirely in **Python.** Main libraries used:

- **Polars**: Extremely fast, multi-threaded DataFrame library (replaces Pandas).
- **DuckDB**: in-process analytical SQL database. Used to query the Gold Parquet files directly from disk into the dashboard.
- **Streamlit**: Pure Python web framework used to build the interactive dashboard.
- **Plotly**: Interactive charting library used for the dashboard's visualizations.

Pipeline Architecture



Data Quality

- **The Quarantine Layer (Rows Removed)**:
  - Missing Critical Keys (Null PLN or Null ACTUAL sales).
  - Header Pollution (Embedded CSV headers like PLN == "PLN").
- **Missing Value Imputation (Data Kept)**:
  - Null aliases ("na", "none", "nobusinessunit") standardized.
  - Missing PROMO values safely default to "N".
  - Missing Categoricals (BU, SEG, Categories) default to "Unknown".
- **Data Polishing**:
  - Dates converted from 1YYMMDD to standard dates.
  - Prefixes stripped from the segment ("NoSegment-A" -> "A").
  - Internal acronyms translated ("NVLTY/GUM/MINT" -> "Gum & Mints").

```python
# 4. Validation Logic (Quarantine Layer tagging)
clean_df = clean_df.with_columns(
    pl.when(pl.col("PLN").cast(pl.String) == "PLN").then(pl.lit("Header Pollution"))
    .when(pl.col("PLN").is_null() | pl.col("ACTUAL").is_null()).then(pl.lit("Missing Critical Keys"))
    .otherwise(pl.lit("Valid"))
    .alias("ERROR_REASON")
)
```

```python
# Drop quarantine rows to free up memory
valid_df = clean_df.filter(pl.col("ERROR_REASON") == "Valid").drop("ERROR_REASON")

# 5. Deduplicate SECOND (Faster because dataset is smaller now)
valid_df = valid_df.unique()

# 6. Standardize Missing/Null Values BEFORE building the Golden Record
null_aliases = ["na", "none", "unknown", "nan", "null", "", "nobusinessunit"]
valid_df = valid_df.with_columns(
    pl.when(cs.string().str.to_lowercase().str.strip_chars().is_in(null_aliases))
    .then(pl.lit(None))
    .otherwise(cs.string())
    .name.keep()
)
```

The Raw Data (Before and After)

- **Legacy Date Formats:** The WEEK column uses a legacy format like 1160529. Converting it into standard, queryable Date objects so our time-series charts actually work.
- **Dirty Categoricals:** We have dirty entries like NoSegment-C. The pipeline automatically sanitizes these inconsistencies.
- **Schema Artifacts:** Column headers were messy - the PROMO column came in as PROMO,, with trailing commas.

BEFORE (Raw CSV Mess):
shape: (1, 9)

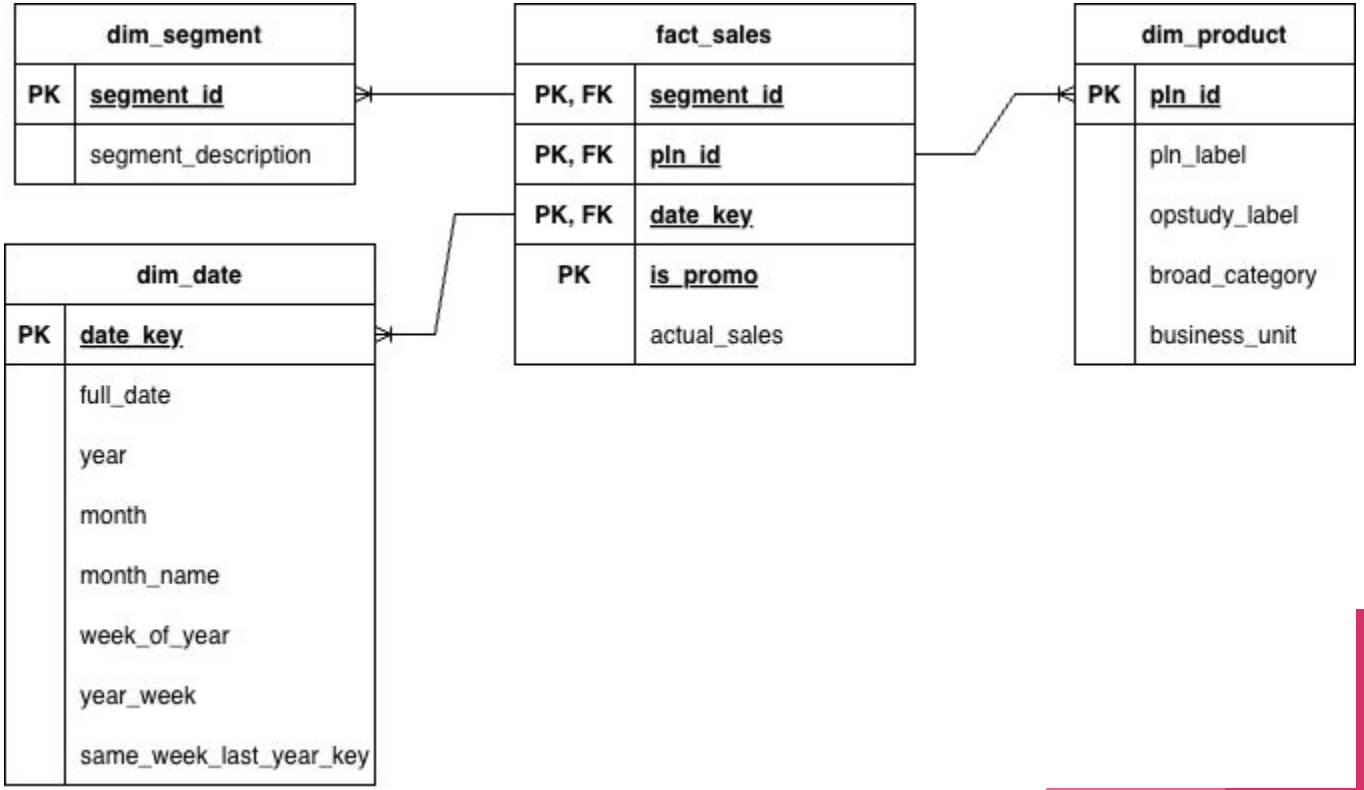
WEEK	PLN	PLN_LABEL	PRODUCT_CATEGORY_LABEL	OPSTUDY_LABEL	BU	ACTUAL	SEG	PROMO,,
---	---	---	---	---	---	---	---	---
str	str	str	str	str	str	f64	str	str
"1231015"	1001	" mEsSy ItEm NaMe "	" cHoCoLaTe "	NVLTY/GUM/MINT	none	10.5	NoSegment-A	"Y",,

AFTER (Cleaned, Standardized, & Typed Silver Layer):
shape: (1, 9)

PLN	ACTUAL	SEG	PROMO	DATE	PLN_LABEL	BU	PRODUCT_CATEGORY_LABEL	OPSTUDY_LABEL
---	---	---	---	---	---	---	---	---
str	f32	cat	cat	date	cat	cat	cat	cat
1001	10.5	A	Y	2023-10-15	MESSY ITEM NAME	Unknown	CHOCOLATE	Gum & Mints

Image: example of what happens to rows from Bronze to Silver layer. Date is parsed, strings are capitalized/stripped, nulls are standardized, acronyms polished, and types optimized

Gold Layer: Star Schema



Gold Layer Engine (Why Star Schema and DuckDB?)

- **Star Schema removes redundancy and optimizes storage**
- **DuckDB** is an in-process analytical database (**OLAP**) that queries these **Parquet** files **directly without loading them into memory**. Instead of writing complex, slow Python code in our Streamlit app, **we push all the heavy aggregations down to the DuckDB SQL engine.**

```sql
SELECT
    DATE,
    CURRENT_SALES,
    LAG(CURRENT_SALES, 1) OVER (ORDER BY DATE) AS PREV_SALES,
    LAG(CURRENT_SALES, 52) OVER (ORDER BY DATE) AS PREV_YOY_SALES
FROM weekly_sales
```

Example of the Instant Year-over-Year calculations using LAG window function

Gold layer: Categorization Engine

- **The Problem:** ~1500 product categories, a lot of them with similar meanings (e.g., "VITAMINS & SUPPLEMENTS", "COUGH & COLD", "PPIS"). Having them on the Executive Dashboard would only clatter the plots.
- **The Solution:** A Python-based Rules Engine mapping to a few High-Level Domains.
- **Example Flow:**
  - "CHOCOLATE" -> Exact Match -> "Confectionery"
  - "DAILY VITAMINS" -> Keyword Match ("VITAMIN") -> "Health"
  - "UNKNOWN ITEM" -> Fallback -> "Other"

```
EXACT_MATCHES = {
    "CHOCOLATE": "Confectionery",
    "KING CHOCOLATE": "Confectionery",
    "COUGH & COLD": "Health",
    "VITAMINS & SUPPLEMENTS": "Health",
    "LAUNDRY SOAP": "Household"
}
```

```
KEYWORD_MAPPINGS = {
    "Beverages": ["WINE", "BEER", "LIQUOR", "WATER", "COFFEE", "SODA", "JUICE"],
    "Health": ["VITAMIN", "PAIN", "COLD", "ALLERG", "MEDICINE", "PILL", "RX"],
    "Electronics": ["BATTERY", "CABLE", "CHARGER", "USB", "AUDIO", "VIDEO"]
}
```

Steps taken for optimization

```
--- Basic Information ---
Total Rows: 19,326,948
Total Columns: 17
```

- **Using Polars over Pandas:**
  - Multi-threading & Lazy Evaluation (Parallel calculation).
  - Sub-sorting deduplication.
- **Storage (Parquet + Star Schema):**
  - Columnar compression (**94% reduction in storage: ~2 GB** raw CSV **down to ~120 MB** Parquet).
  - Data Type Downcasting: (e.g., 32-bit Integer Keys).
  - Eliminates redundant text strings.
- **Application Layer (DuckDB + Streamlit Caching):**
  - Direct-to-disk querying (Bypasses Python memory limits).
  - @st.cache_data prevents redundant re-calculations.

Dashboard showcase

Thank you for your attention!